

TP M1 E3A Projet TER

Système Embarqué pour Drone Autonome Sans GPS

UNIVERSITÉ ÉVRY PARIS-SACLAY
UFR SCIENCES ET TECHNOLOGIES

Rédigé par
SOUAB Mohamed
CHARTIER Manoah
SCHWALLER Maxime
LITAIZE-RICHERT Clement

10 mai 2026

1 Contexte du Projet

L'objectif principal de notre projet est de concevoir un drone autonome doté de capacités de vision par ordinateur, capable de naviguer et d'opérer dans des environnements dépourvus de signal GPS (*GPS-denied environments*).

Pour pallier l'absence de GPS, le système repose entièrement sur ses capteurs embarqués et sa puissance de calcul. Cela implique la mise en place d'une architecture logicielle capable d'assurer la cartographie de l'environnement (SLAM), l'estimation précise de la trajectoire (VIO) et la reconnaissance de cibles pour des manœuvres complexes comme l'atterrissage de précision.

2 Objectifs et Cahier des Charges

Le cahier des charges de ce projet vise à développer une chaîne complète d'intégration logicielle pour le contrôle autonome d'un drone quadrotor. Le système matériel s'articule autour d'un châssis Holybro X500 V2, d'un contrôleur de vol Pixhawk PX4 et d'un ordinateur compagnon Raspberry Pi 4. L'objectif central est de permettre une navigation fiable dans des environnements sans GPS.

Pour répondre à ce défi technique, les objectifs du projet ont été structurés autour des axes suivants :

- **Simulation et Validation (SITL)** : Valider la logique de vol autonome en simulant et en testant la fiabilité des missions sous Gazebo avant tout essai matériel. Cette étape couple le firmware PX4 et le middleware ROS 2 Jazzy.
- **Abstraction Logicielle et Contrôle (*Offboard*)** : Développer une architecture modulaire de haut niveau (API Python) permettant de masquer la complexité du protocole MAVLink bas niveau. La communication vitale entre l'ordinateur compagnon et le contrôleur de vol est assurée de manière fluide par le paquet MAVROS.
- **Localisation et Cartographie (VIO & SLAM)** : Naviguer en milieu hostile en intégrant l'odométrie visuelle et la cartographie 3D (VSLAM) pour le maintien de position. Ce pipeline exploite une caméra de profondeur (Intel RealSense D455) et l'algorithme RTAB-Map, sélectionné pour sa robustesse.
- **Atterrissage de Précision par Perception Intelligente** : Doter le drone de capacités de vision par ordinateur pour réaliser un atterrissage de précision. Ce module utilise OpenCV et `cv_bridge` pour détecter des marqueurs géométriques (ArUco) et asservir le drone en conséquence.

3 Architecture Logicielle et Abstraction sous ROS 2

3.1 Communication MAVLink et MAVROS

Le contrôleur de vol (Pixhawk PX4) gère la stabilisation et la dynamique de vol en temps réel (bas niveau) en exécutant du code C/C++. L'ordinateur compagnon (Raspberry Pi 4), quant à lui, exécute des algorithmes de traitement lourds (vision, SLAM) sous ROS 2 Jazzy (haut niveau).

Pour faire le pont entre ces deux environnements, le protocole MAVLink est utilisé. L'intégration dans l'écosystème ROS 2 est assurée par le paquet **MAVROS**. Ce dernier agit comme un traducteur bidirectionnel :

- Il convertit les messages MAVLink provenant du PX4 (télémetrie, état de l'IMU, statut de la batterie) en topics ROS 2 standards.
- Il traduit les consignes de position, de vitesse ou d'attitude publiées sur les topics ROS 2 en commandes MAVLink compréhensibles par l'autopilote.

3.2 API d'Abstraction Python (Contrôle Offboard)

Afin de simplifier le développement des missions autonomes et de masquer la complexité inhérente aux échanges MAVLink bruts, une API d'abstraction logicielle a été développée en Python. Cette couche logicielle permet un contrôle du drone de manière totalement transparente, que le système soit exécuté en simulation (SITL sous Gazebo) ou sur le matériel réel.

```
1 import asyncio
2 from mavsdk import System
3 from mavsdk.offboard import (OffboardError, PositionNedYaw)
4 class Drone:
5     def __init__(self, system_address="udp://:14540"):
6         self.system_address = system_address
7         self.drone = System()
8         self._is_connected = False
9     async def connect(self):
10        print(f"--> Connecting to {self.system_address}...")
11        await self.drone.connect(system_address=self.system_address)
12        async for state in self.drone.core.connection_state():
13            if state.is_connected:
14                self._is_connected = True
15                print("--> Drone Connected!")
16                break
17    async def check_health(self):
18        print("--> Checking system health...")
19        async for health in self.drone.telemetry.health():
20            if not health.is_global_position_ok:
21                print("!! GPS Error: No Global Position fix.")
22                return False
23            break
24        async for battery in self.drone.telemetry.battery():
25            if battery.remaining_percent < 0.2:
26                print(f"!! Low Battery: {battery.remaining_percent*100:.1f}%")
27                return False
28            break
29        print("--> Systems Green.")
30        return True
31    async def arm_and_takeoff(self, altitude_meters):
32        print("--> Arming...")
33        await self.drone.action.arm()
34        print(f"--> Taking off to {altitude_meters}m...")
35        await self.drone.action.set_takeoff_altitude(altitude_meters)
36        await self.drone.action.takeoff()
37        target_reached = False
38        async for position in self.drone.telemetry.position():
39            current_altitude = position.relative_altitude_m
40            print(f"    Altitude: {current_altitude:.2f}m / {altitude_meters}m")
41            if current_altitude >= altitude_meters * 0.95:
42                target_reached = True
43                print("--> Takeoff complete.")
44                break
45            await asyncio.sleep(0.5)
46        if not target_reached:
47            print("!! Warning: Takeoff loop exited without reaching target")
48    async def goto_local(self, north, east, down, yaw):
49        print(f"--> Moving to N:{north} E:{east} D:{down} Y:{yaw}")
50        try:
51            await self.drone.offboard.set_position_ned(
52                PositionNedYaw(north, east, down, yaw)
53            )
54            if not await self.ensure_offboard_active():
```

```

55         await self.drone.offboard.start()
56     except OffboardError as e:
57         print(f"!! Offboard Error: {e}")
58     async def ensure_offboard_active(self):
59         return self.drone.offboard.is_active()
60     async def land(self):
61         print("--> Landing...")
62         try:
63             await self.drone.offboard.stop()
64         except:
65             pass
66         await self.drone.action.land()
67         print("--> Landing initiated.")

```

4 Cartographie et Odométrie Visuelle-Inertielle (SLAM & VIO)

Dans un environnement *GPS-denied*, le drone perd sa source absolue de positionnement global. Pour maintenir un vol stable et naviguer de manière autonome, le système doit estimer sa propre position à partir de ses capteurs embarqués (Odométrie) et construire une représentation spatiale de son environnement (Cartographie). Ce double défi est relevé grâce à une approche couplant VIO et SLAM.

4.1 Pipeline VIO avec Intel RealSense D455

L'odométrie visuelle-inertielle (VIO) est la première brique de notre chaîne de perception. Elle repose sur la caméra de profondeur Intel RealSense D455, qui fournit des flux d'images stéréo, des cartes de profondeur et des données inertielles à haute fréquence via son IMU intégré.

Le wrapper ROS 2 de la caméra traite ces données pour estimer le déplacement relatif du drone entre deux instants. Cette estimation locale génère une odométrie continue et fluide, indispensable pour boucler la boucle d'asservissement en vitesse et en position du Pixhawk via MAVROS. Lors de nos essais, l'intégration de la vision avec l'estimateur d'état (EKF2) du contrôleur de vol s'est révélée stable et fiable.

4.2 Cartographie 3D avec RTAB-Map

Si la VIO permet d'estimer la trajectoire locale, elle est sujette à une dérive temporelle (*drift*). Pour corriger cette dérive et générer une carte exploitable pour la planification de trajectoire, nous avons intégré l'algorithme **RTAB-Map** (Real-Time Appearance-Based Mapping).

Le choix de RTAB-Map s'est imposé suite à une analyse comparative rigoureuse. En effet, lors du développement, RTAB-Map a prouvé sa robustesse face aux instabilités de calibration fréquemment rencontrées sur des algorithmes alternatifs tels que VINS-Fusion ou ORB-SLAM.

RTAB-Map utilise les images RGB-D de la RealSense pour extraire des caractéristiques visuelles, détecter des fermetures de boucles (*loop closures*) et optimiser le graphe de pose global. Le nœud génère simultanément :

- Un nuage de points 3D de l'environnement, visualisable sous RViz pour le monitoring.
- Une carte d'occupation 2D (*Occupancy Grid*), qui peut être exploitée par le planificateur de navigation pour l'évitement d'obstacles.

4.3 L'Arbre des Transformations (TF Tree)

Dans ROS 2, la cohérence spatiale entre les différents capteurs et algorithmes est garantie par l'arbre des transformations (TF2). Pour ce projet, la chaîne cinématique standard est définie de la manière suivante :

1. **map** : Le repère global statique, initialisé au point de décollage. RTAB-Map publie la transformation entre **map** et **odom** pour corriger la dérive.

2. `odom` : Le repère local d'odométrie, généré par le pipeline VIO de la caméra RealSense.
3. `base_link` : Le centre de gravité du drone.
4. `camera_link` : Le repère physique de la caméra RealSense, dont la transformation statique par rapport à `base_link` est définie par un nœud ROS 2 `static_transform_publisher`.

5 Guide Pratique : Contrôle et Communication avec le Drone

5.1 Installation du Firmware PX4 et Dépendances

Le développement s'effectue nativement sur un environnement Linux (Ubuntu). La première étape pour les futurs étudiants est de récupérer le code source officiel de l'autopilote PX4 et d'installer la chaîne de compilation croisée ainsi que le simulateur Gazebo.

Ouvrez un terminal et exécutez les commandes suivantes pour cloner le dépôt et lancer le script d'installation automatisé :

```
1 # Créer un espace de travail et s'y déplacer
2 mkdir -p ~/workspace
3 cd ~/workspace
4
5 # Cloner le depot officiel PX4-Autopilot
6 git clone https://github.com/PX4/PX4-Autopilot.git
7
8 # Se déplacer dans le repertoire clone
9 cd PX4-Autopilot
10
11 # Exécuter le script d'installation des dependances pour Ubuntu
12 # (Installe les compilateurs, Gazebo, et configure les permissions)
13 bash ./Tools/setup/ubuntu.sh
```

Note : Le script `ubuntu.sh` modifie les groupes d'utilisateurs de votre système (notamment pour l'accès aux ports série `diout`). Un redémarrage complet de l'ordinateur est requis après l'exécution de ce script pour que les changements prennent effet.

5.2 Compilation et Lancement de Gazebo

Une fois l'ordinateur redémarré, le système de compilation du PX4 (basé sur `make`) permet de générer un firmware adapté à une exécution locale (architecture POSIX) et de lancer simultanément le simulateur 3D.

```
1 # Se placer dans le repertoire source du firmware
2 cd ~/workspace/PX4-Autopilot
3
4 # Compiler le firmware et lancer la simulation SITL avec Gazebo
5 make px4_sitl gazebo
```

Lors de l'exécution de cette commande, le processus suivant se met en place :

1. Le terminal actif se transforme en console système (NuttShell / `pxh`) du contrôleur PX4 virtuel.
2. L'interface graphique de Gazebo s'ouvre, chargeant le modèle 3D du drone (par défaut un quadrotor) sur une piste de décollage virtuelle.

5.3 Topologie Réseau et Routage MAVLink

Pour interagir avec ce drone virtuel, il est crucial de comprendre comment le simulateur gère les communications. Le PX4 simulé expose automatiquement plusieurs ports UDP sur la boucle locale (127.0.0.1) pour diffuser les messages MAVLink :

- **Port UDP 14550** : Réservé à la télémétrie standard et aux stations de contrôle. Si vous lancez le logiciel **QGroundControl** sur la même machine, il s’y connectera automatiquement pour afficher la carte, l’horizon artificiel et les paramètres du drone.
- **Port UDP 14540** : Dédié au contrôle *Offboard*. C’est sur ce port que nos futurs scripts Python (MAVSDK) ou C viendront se connecter pour envoyer des consignes de trajectoire et des commandes d’armement.
- **Port UDP 14557** : Port à haut débit utilisé pour relier le contrôleur de vol à la couche d’abstraction ROS 2 via MAVROS (abordé dans les sections suivantes).

Dès lors que Gazebo affiche le drone et que QGroundControl annonce un statut *"Ready to Fly"*, l’environnement de simulation de base est pleinement opérationnel.

5.4 Contrôle Bas Niveau en C (MAVLink Direct)

Le code source situé dans le dossier `workspace/src/c/` illustre la communication fondamentale avec le drone. Ces scripts n’utilisent aucune bibliothèque tierce complexe autre que les en-têtes MAVLink bruts. Ils ouvrent un socket UDP sur le port 14540 et envoient directement des structures de données binaires (comme `MAV_CMD_NAV_TAKEOFF`).

5.4.1 Compilation des exécutables C

Pour compiler ces fichiers, il est nécessaire d’inclure le chemin vers les bibliothèques MAVLink. Depuis votre terminal, naviguez vers le dossier source C du projet et exécutez les commandes `gcc` suivantes :

```
1  # Se placer dans le repertoire source C
2  cd workspace/src/c/
3
4  # Compiler la commande de decollage
5  gcc -I../libs/mavlink -o ../bin/simple_takeoff simple_takeoff.c
6
7  # Compiler la commande d'atterrissage
8  gcc -I../libs/mavlink -o ../bin/simple_landing simple_landing.c
9
10 # Compiler la commande Return-To-Launch (RTL)
11 gcc -I../libs/mavlink -o ../bin/rethome rethome.c
12
13 # Compiler la mission de navigation par points de passage (cœur)
14 gcc -I../libs/mavlink -o ../bin/waypoints waypoints.c -lm
```

Note : Le flag `-lm` est parfois requis pour lier la bibliothèque mathématique de C si des fonctions comme `NAN` ou `math.h` sont sollicitées lors de la compilation.

5.4.2 Exécution des commandes C

Une fois que le contrôleur de vol (en simulation SITL ou sur le matériel réel) est prêt à recevoir des commandes, vous pouvez exécuter les binaires générés. Placez-vous dans le répertoire `bin` :

```
1  cd ../bin/
2
```

```
3 # Lancer le decollage (le drone s'arme et monte a 10 metres)
4 ./simple_takeoff
5
6 # Demander l'atterrissage immediat
7 ./simple_landing
```

5.5 Abstraction Haut Niveau en Python (MAVSDK)

Bien que le C soit formateur pour comprendre le fonctionnement intime du protocole MAVLink, l'orchestration de missions autonomes complexes (nécessitant une synchronisation avec la vision par ordinateur ou le SLAM) exige un langage plus flexible. L'API développée dans `workspace/src/python/drone_interface.py` répond à ce besoin en encapsulant la complexité via la bibliothèque **MAVSDK** et l'asynchronisme (`asyncio`).

5.5.1 Lancement des scripts Python

Contrairement au C, les scripts Python ne nécessitent pas de compilation. Assurez-vous simplement que la dépendance `mavsdk` est installée sur votre système hôte (`pip install mavsdk`), puis exécutez les scripts directement depuis le terminal.

```
1 # Se placer dans le repertoire source Python
2 cd workspace/src/python/
3
4 # Executer le script de decollage autonome (inclus les pre-flight checks)
5 python3 takeoff.py
6
7 # Executer la commande d'atterrissage
8 python3 landing.py
```

5.5.2 Développement de nouvelles missions

Pour concevoir de nouveaux comportements de vol, il suffit d'importer la classe `Drone` depuis le module `drone_interface`. Voici le squelette de code asynchrone minimal à respecter pour tout nouveau script de mission :

```
1 import asyncio
2 from drone_interface import Drone
3
4 async def mission_personnalisee():
5     # Initialisation de l'instance du drone
6     drone = Drone(system_address="udp://:14540")
7     await drone.connect()
8
9     # Verification des systemes vitaux avant le decollage
10    if await drone.check_health():
11        print("Parametres nominaux. Debut de la mission.")
12
13        # Exemple : Decollage a 5 metres d'altitude
14        await drone.arm_and_takeoff(altitude_meters=5)
15
16        # [Ajouter ici la logique de deplacement via goto_local]
17
18        # Atterrissage une fois la mission terminee
19        await drone.land()
20    else:
```

```
21         print("Erreur systeme : atterrissage/desarmement force.")
22
23 if __name__ == "__main__":
24     asyncio.run(mission_personnalisee())
```

6 Intégration de ROS 2 et Exécution des Missions Autonomes

6.1 Installation Native de ROS 2 Jazzy

ROS 2 Jazzy est la distribution LTS (Long Term Support) recommandée pour Ubuntu 24.04. L'installation s'effectue via les dépôts officiels de paquets Debian.

Ouvrez un terminal et exécutez les commandes suivantes pour configurer les sources et installer la version *Desktop* :

```
1  # Mettre a jour l'index des paquets
2  sudo apt update && sudo apt install locales
3  sudo locale-gen en_US en_US.UTF-8
4  sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
5  export LANG=en_US.UTF-8
6
7  # Ajouter le depot ROS 2
8  sudo apt install software-properties-common
9  sudo add-apt-repository universe
10 sudo apt update && sudo apt install curl -y
11 sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
   ↪ /usr/share/keyrings/ros-archive-keyring.gpg
12 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
   ↪ http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee
   ↪ /etc/apt/sources.list.d/ros2.list > /dev/null
13
14 # Installer ROS 2 Jazzy et les outils de developpement
15 sudo apt update
16 sudo apt install ros-jazzy-desktop python3-colcon-common-extensions -y
17
18 # Sourcer l'environnement ROS 2 automatiquement a l'ouverture du terminal
19 echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
20 source ~/.bashrc
```

6.2 Configuration de l'Espace de Travail (Workspace) et MAVROS

Une fois ROS 2 installé, il est nécessaire de créer un espace de travail (*workspace*) dédié à notre projet. Cet espace accueillera les nœuds de perception (OpenCV, RTAB-Map) et le paquet **MAVROS**, qui agit comme un pont de communication entre les topics ROS 2 et les messages MAVLink du contrôleur de vol PX4.

```
1  # Creation de l'architecture du workspace
2  mkdir -p ~/workspace/src
3  cd ~/workspace
4
5  # Installation de MAVROS pour ROS 2 Jazzy
6  sudo apt install ros-jazzy-mavros ros-jazzy-mavros-extras -y
7
8  # Telechargement des scripts de geoids (requis pour MAVROS)
9  wget -O - https://raw.githubusercontent.com/mavlink/mavros/master/mavros/scripts/install_geographiclib_d
   ↪ atatasets.sh | sudo bash
```

```
10
11 # Compilation du workspace vide (pour initialiser l'environnement local)
12 colcon build --symlink-install
13 echo "source ~/workspace/install/setup.bash" >> ~/.bashrc
14 source ~/.bashrc
```

7 Exécution des Nœuds ROS 2 et Atterrissage de Précision

Une fois l'environnement de simulation SITL lancé et la connexion MAVLink établie, nous pouvons exploiter la couche d'autonomie développée sous ROS 2 Jazzy. L'ensemble des scripts de perception (vision par ordinateur) et de navigation (contrôle de trajectoire) est centralisé au sein du paquet `drone_control`.

7.1 Compilation du Paquet `drone_control`

Contrairement aux scripts Python autonomes utilisant MAVSDK, les nœuds ROS 2 nécessitent d'être indexés et construits au sein de l'espace de travail via l'outil `colcon`. Ouvrez un nouveau terminal et exécutez la séquence suivante :

```
1 # Se placer a la racine de l'espace de travail ROS 2
2 cd ~/workspace/
3
4 # Compiler le paquet (l'option symlink-install evite de devoir
5 # recompiler a chaque modification d'un script Python)
6 colcon build --symlink-install
7
8 # Sourcer l'environnement local pour rendre les executables disponibles
9 source install/setup.bash
```

7.2 Validation des Comportements Dynamiques (*Offboard*)

```
1 # 1. Test de maintien de position simple (Hover au-dessus du point de decollage)
2 ros2 run drone_control hover
3
4 # 2. Navigation spatiale par points de passage (Trajectoire en carre)
5 ros2 run drone_control square
6
7 # 3. Asservissement en vitesse (Trajectoire circulaire fluide)
8 ros2 run drone_control circle
9
10 # 4. Controle relatif dans le repere du drone (Body Frame)
11 ros2 run drone_control body
```

7.3 Déploiement de l'Atterrissage Autonome par Vision (ArUco)

L'aboutissement de notre chaîne d'intégration est l'atterrissage de précision. Ce processus complexe nécessite la synchronisation de trois sous-systèmes logiciels distincts fonctionnant en parallèle :

- **Le pont MAVROS** : Assure la transmission des consignes de vol bas niveau.
- **Le nœud de perception (`vision_aruco.py`)** : S'abonne au flux vidéo de la caméra (`/camera/image_raw`), traite l'image avec OpenCV via `cv_bridge`, détecte la matrice du marqueur ArUco, et publie un vecteur d'erreur normalisé sur le topic `/landing/target`.

- **Le nœud de contrôle** (`control_land.py`) : Interprète l'erreur visuelle pour générer des consignes de vitesse inversées (régulateur P) afin de centrer le drone, et déclenche l'atterrissage forcé (*AUTO.LAND*) lorsque l'altitude critique est atteinte.

Pour orchestrer ce déploiement sans avoir à ouvrir de multiples terminaux, un fichier de lancement (*launch file*) a été rédigé. Il initialise l'ensemble de la pile logique avec les bons paramètres.

```
1 # Lancer la sequence complete d'atterrissage de precision (MAVROS + Vision + Controle)
2 ros2 launch drone_control aruco_landing.launch.py
```

*Note : Le fichier `aruco_landing.launch.py` intègre une temporisation (*TimerAction*). Le nœud de contrôle de vol attendra 3 secondes avant de s'exécuter. Ce délai garanti que MAVROS ait le temps d'établir une connexion stable avec le contrôleur Pixhawk avant toute requête d'armement.*

8 Navigation en Environnement Sans GPS (*GPS-Denied*)

Dans une configuration standard, le contrôleur de vol PX4 repose massivement sur le signal GPS pour alimenter son Filtre de Kalman Étendu (EKF2). Le GPS fournit une estimation absolue de la position et de la vitesse. En l'absence de ce signal (vol en intérieur, zones urbaines denses, brouillage), le drone perd son repère global et le PX4 interdit par sécurité l'activation des modes de vol autonomes (comme le mode *Offboard*).

Pour pallier cette absence et faire fonctionner le drone en environnement *GPS-denied*, il est nécessaire de substituer les données satellitaires par des données de perception locale générées par les capteurs embarqués.

8.1 Odométrie Visuelle-Inertielle (VIO)

La première étape de notre solution repose sur l'Odométrie Visuelle-Inertielle (VIO). Cette technique utilise les données de la caméra Intel RealSense D455 embarquée sur le drone.

La caméra combine deux types de flux :

- **Flux visuel (Caméras Stéréo/Profondeur)** : Permet de suivre le déplacement des pixels (caractéristiques de l'image) d'une trame à l'autre pour déduire le mouvement relatif.
- **Flux inertiel (IMU interne à la caméra)** : Mesure les accélérations et les vitesses angulaires à très haute fréquence, comblant les "trous" entre deux images vidéo.

Une fois traitée par la pile ROS 2 sur le Raspberry Pi 4, cette estimation de pose locale est injectée dans le PX4 via MAVROS. L'intégration de la vision avec l'estimateur d'état (EKF2) du PX4 s'est avérée particulièrement stable lors de nos essais.

8.2 Configuration de l'Estimateur d'État (EKF2) du PX4

Pour que le PX4 accepte cette odométrie visuelle externe au lieu de chercher désespérément un signal GPS, une reconfiguration stricte de ses paramètres internes via *QGroundControl* (ou via le shell MAVLink) est obligatoire.

Voici les paramètres fondamentaux à modifier pour autoriser le vol *GPS-denied* par vision :

```
1 # Desactiver l'exigence du GPS
2 SYS_HAS_GPS = 0
3
4 # Configurer l'EKF2 pour utiliser la vision externe (EV - External Vision)
5 EKF2_EV_CTRL = 15 # Active la fusion de position horizontale, verticale, yaw et vitesse
6 EKF2_HGT_REF = 3  # Definit la vision comme source principale de hauteur (au lieu du barometre)
```

```
7 EKF2_GPS_CTRL = 0 # Desactive completement la fusion GPS dans le filtre
8
9 # Rehausser les seuils de delai pour eviter des failsafes intempestifs
10 EKF2_EV_DELAY = 175 # Ajuster selon la latence de calcul du Pi 4 (en millisecondes)
```

Avec ces paramètres, lorsque MAVROS publie l'odométrie générée par la caméra RealSense sur le topic `/mavros/vision_pose/pose`, le PX4 reconstruit son repère local NED (North-East-Down) et autorise le déclenchement de notre script Python (mode *Offboard*).

8.3 VSLAM et Cartographie Locale avec RTAB-Map

Si la VIO garantit la stabilité immédiate du drone, elle souffre inévitablement d'une dérive cumulative (*drift*) sur le long terme : une erreur de quelques millimètres à chaque seconde se transforme rapidement en plusieurs mètres.

Pour corriger ce phénomène et véritablement naviguer en milieu hostile, le système intègre la brique logicielle **RTAB-Map** (VSLAM). Cet algorithme a pour but d'intégrer l'odométrie visuelle et la cartographie 3D pour assurer le maintien précis de la position.

RTAB-Map analyse les images et enregistre des "signatures" visuelles de l'environnement. Lorsqu'il reconnaît un lieu déjà visité, il effectue une *fermeture de boucle* (Loop Closure) et corrige rétroactivement toute la trajectoire déviée.

9 Guide de Reproduction de la Simulation Complète (VSLAM et PX4)

Cette section détaille la procédure exacte pour instancier l'environnement de simulation complet. Ce déploiement met en œuvre le firmware PX4 (SITL), le simulateur physique Gazebo, les ponts de communication ROS 2, et l'algorithme RTAB-Map pour générer une odométrie visuelle en temps réel. Cette configuration simule les conditions d'un vol en environnement *GPS-denied*.

9.1 Installation des Dépendances et Prérequis

Avant de lancer la simulation, il est impératif d'installer les paquets ROS 2 nécessaires à la communication spatiale et à la cartographie. Ces commandes doivent être exécutées sur la machine hôte (Ubuntu 24.04 avec ROS 2 Jazzy).

```
1 # Mise a jour de l'index des paquets
2 sudo apt update
3
4 # Installation de MAVROS pour la communication avec le PX4
5 sudo apt install ros-jazzy-mavros ros-jazzy-mavros-extras -y
6
7 # Installation du pont de communication entre ROS 2 et Gazebo
8 sudo apt install ros-jazzy-ros-gz-bridge -y
9
10 # Installation de RTAB-Map pour le VSLAM
11 sudo apt install ros-jazzy-rtabmap-ros -y
12
13 # Installation des outils de manipulation de topics (pour le relais d'odometrie)
14 sudo apt install ros-jazzy-topic-tools -y
```

Le bon fonctionnement de MAVROS repose sur des bibliothèques géospatiales spécifiques. Le script suivant télécharge et installe ces jeux de données :

```
1 wget -O - https://raw.githubusercontent.com/mavlink/mavros/master/mavros/scripts/install_geographiclib_d
↪ atassets.sh | sudo bash
```

9.2 Séquence de Lancement Multi-Terminaux

L'architecture distribuée de ROS 2 nécessite l'exécution simultanée de plusieurs nœuds. Il est recommandé d'utiliser un multiplexeur de terminaux (comme `tmux`) pour gérer ces processus.

Rappel : L'environnement ROS 2 (`source /opt/ros/jazzy/setup.bash`) doit être sourcé dans chaque nouveau terminal.

9.2.1 Terminal 1 : PX4 SITL et Gazebo

Cette commande compile le firmware et lance Gazebo en instanciant le modèle de drone x500 équipé d'une caméra de profondeur.

```
1 cd ~/workspace/PX4-Autopilot
2 make px4_sitl gz_x500_depth
```

9.2.2 Terminal 2 : Liaison MAVROS

Initialisation du pont MAVLink entre le middleware ROS 2 et le port haut débit du contrôleur de vol simulé.

```
1 ros2 launch mavros px4.launch fcuk_url:=udp://:14540@127.0.0.1:14557
```

9.2.3 Terminal 3 : Synchronisation Temporelle (Clock Bridge)

Pour garantir la cohérence des calculs de RTAB-Map, l'horloge de ROS 2 doit être synchronisée avec le temps simulé par Gazebo.

```
1 ros2 run ros_gz_bridge parameter_bridge \
2   "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock" \
3   --ros-args -p use_sim_time:=true
```

9.2.4 Terminal 4 : Flux Vidéo et Caméra (Camera Bridge)

Importation des flux simulés (image RGB, informations de calibration et carte de profondeur) depuis l'environnement Gazebo vers les topics ROS 2 standards.

```
1 ros2 run ros_gz_bridge parameter_bridge \
2   /world/default/model/x500_depth_0/link/camera_link/sensor/IMX214/image@sensor_msgs/msg/Image[gz.msgs.I
↪   mage \
3   /world/default/model/x500_depth_0/link/camera_link/sensor/IMX214/camera_info@sensor_msgs/msg/CameraInf
↪   o[gz.msgs.CameraInfo \
4   /depth_camera@sensor_msgs/msg/Image[gz.msgs.Image
```

9.2.5 Terminal 5 : Arbre des Transformations (TF2)

Définition de la transformation statique (rotation et translation) entre le centre de gravité du drone (`base_link`) et le capteur optique (`camera_link`).

```
1 ros2 run tf2_ros static_transform_publisher 0.1 0 0 -1.5708 0 -1.5708 base_link camera_link
```

9.2.6 Terminal 6 : Cartographie VSLAM (RTAB-Map)

Lancement de l'algorithme RTAB-Map en lui assignant les topics vidéo fraîchement importés de Gazebo. Le paramètre `use_sim_time` est crucial.

```
1 ros2 launch rtabmap_launch rtabmap.launch.py \  
2   rtabmap_args:="--delete_db_on_start" \  
3   rgb_topic:=/world/default/model/x500_depth_0/link/camera_link/sensor/IMX214/image \  
4   depth_topic:=/depth_camera \  
5   camera_info_topic:=/world/default/model/x500_depth_0/link/camera_link/sensor/IMX214/camera_info \  
6   frame_id:=base_link \  
7   approx_sync:=true \  
8   use_sim_time:=true
```

9.2.7 Terminal 7 : Relais de l'Odométrie vers PX4

Enfin, l'odométrie visuelle calculée par RTAB-Map doit être réinjectée dans le contrôleur de vol pour alimenter son filtre EKF2, autorisant ainsi la navigation sans GPS.

```
1 ros2 run topic_tools relay /rtabmap/odom /mavros/odometry/out
```

Une fois cette séquence complétée, le système est stabilisé. Les scripts de contrôle *Offboard* (Python/MAVSDK) peuvent alors être exécutés pour asservir le quadrotor.